

Inputs and notation (once, up front)

- **D**: training set with n labeled records (x_j, y_j) .
- **X**: a **categorical** attribute with domain $\text{dom}(X) = \{v_1, \dots, v_m\}$.
- **k**: number of classes $\{c_1, \dots, c_k\}$.
- **l**: max subset size to consider for the split $X \in V$ (controls search).
- n_{vi} : count of records with $X = v$ and class c_i .
- D_Y : the partition where $X \in V$; D_N : the complement where $X \notin V$.
- Class PMFs used by Gain come from the frequency table n_{vi} via Eqs. (19.21) and (19.22).

Lines 1–2: `for i = 1..k do n_i ← 0`

Initialize per-class totals n_i . (Many implementations also accumulate n_i during line 5. While the gain formulas below don't require n_i explicitly, having class totals on hand is standard and useful for other criteria.)

Line 3: `forall v ∈ dom(X) do n_vi ← 0`

Initialize the **category–class contingency table**. Conceptually this is for every pair (v, i) ; i.e., you're setting a $m \times k$ table to zero so it can be filled with counts.

Lines 4–5: `for j = 1..n ... update n_vi`

Single pass over the data: for each record, look at its category x_j and class y_j , then increment the corresponding cell n_{vi} . After this loop, the table n_{vi} holds **all frequency statistics** needed for the PMFs. (If desired, you can also update $n_i = \sum_v n_{vi}$ here.)

Line 6: `V* ← ∅; score* ← 0`

Set up trackers for the best split seen so far: V^* (subset of categories that go left) and its score. With **information gain**, higher is better, so initializing `score*` to 0 is fine. (If you were minimizing an impurity, you'd initialize differently.)

Line 7: `forall V ⊂ dom(X), 1 ≤ |V| ≤ l`

Enumerate candidate **subsets of categories** that define the left branch D_Y .

- l controls computational cost:
 - $l = 1$: only singleton splits $X \in \{v\}$.
 - $l = \lfloor m/2 \rfloor$: all distinct binary partitions.
In practice, you may use heuristics (e.g., order categories by class skew, then try prefixes) to avoid exhaustive enumeration when m is large.

Lines 8–9: compute $\hat{P}(c_i \mid D_Y)$

$$\hat{P}(c_i \mid D_Y) = \frac{\sum_{v \in V} n_{vi}}{\sum_{j=1}^k \sum_{v \in V} n_{vj}}$$

This is Eq. (19.21): the **class PMF on the left partition**.

- Numerator: how many records in D_Y belong to class c_i .
- Denominator: total size of D_Y (sum over all classes).

Implementation tip: precompute $s_V(i) = \sum_{v \in V} n_{vi}$ and $S_V = \sum_i s_V(i)$.

Line 10: compute $\hat{P}(c_i \mid D_N)$

$$\hat{P}(c_i \mid D_N) = \frac{\sum_{v \notin V} n_{vi}}{\sum_{j=1}^k \sum_{v \notin V} n_{vj}}$$

This is Eq. (19.22): the **class PMF on the right partition**.

- You can compute complements efficiently via totals:

$$\sum_{v \notin V} n_{vi} = n_i - \sum_{v \in V} n_{vi}, \text{ and similarly for the denominator.}$$

Line 11: `score( $X \in V$ ) ← Gain( $D, D_Y, D_N$ )`

Evaluate the split quality using **information gain** (Eq. 19.5).

- Compute entropy of D, D_Y, D_N from the PMFs above;
- Then $\text{Gain} = H(D) - \frac{|D_Y|}{|D|} H(D_Y) - \frac{|D_N|}{|D|} H(D_N)$.

(You could plug in Gini or CART instead; then you'd choose the **minimum** score.)

Lines 12–13: **keep the best subset**

If the current split's score is better than `score*`, update V^* and `score*`. This retains the **argmax** over all tested subsets.

Line 14: `return ( $V^*$ , score*)`

Output the **best category subset** V^* for splitting X and its score. The tree builder will create two branches: **left** for $X \in V^*$ and **right** for $X \notin V^*$.